

```

        if (!isLeapYear(year)) {
            if (after <= reverseMonthStacks[month]) {
                while (after > months[month]) {
                    after -= months[month];
                }
                return Integer.toString(year * 10000 + month * 100 + after);
            }
            if (after <= reverseMonthStacksLeapYear[month]) {
                while (after > monthsLeapYear[month]) {
                    after -= monthsLeapYear[month];
                }
                return Integer.toString(year * 10000 + month * 100 + after);
            }
        }
        while (after > 365 + getLeapYearCount(year)) {
            after -= 365 + getLeapYearCount(year);
            year++;
        }
        if (!isLeapYear(year)) {
            while (after > months[month]) {
                after -= months[month];
                month++;
            }
            return Integer.toString(year * 10000 + month * 100 + after);
        }
        after -= reverseMonthStacksLeapYear[month];
        year++;
        month = 1;
    }
    while (after > 365 + getLeapYearCount(year)) {
        after -= 365 + getLeapYearCount(year);
        year++;
    }
    if (!isLeapYear(year)) {
        while (after > months[month]) {
            after -= months[month];
            month++;
        }
        return Integer.toString(year * 10000 + month * 100 + after);
    }
    if (after <= reverseMonthStacksLeapYear[month]) {
        while (after > monthsLeapYear[month]) {
            after -= monthsLeapYear[month];
            month++;
        }
        return Integer.toString(year * 10000 + month * 100 + after);
    }
}

private static String getDateDaysBefore(int beforeVal, int yearVal, int monthVal, int
dayVal) {
    int before = beforeVal;
    int year = yearVal;
    int month = monthVal;
    int day = dayVal;
    if (!isLeapYear(year)) {
        if (before < day) {
            return Integer.toString(year * 10000 + month * 100 + day - before);
        }
        before -= day;
        month--;
        if (month == 0) {
            year--;
            month = 12;
        }
    }
    else {
        if (before < day) {
            return Integer.toString(year * 10000 + month * 100 + day - before);
        }
        before -= day;
        month--;
        if (month == 0) {
            year--;
            month = 12;
        }
    }
}

```

```

private static String[] getWeekDays(int year, int month, int day) {
}

private static int getDayOfTheWeek(int year, int month, int day) {
    Calendar cal = Calendar.getInstance();
    cal.set(year, month - 1, day);
    return cal.get(7);
}

private static String getDateDaySDiff(int diff, int year, int month, int day) {
    if (diff > 0)
        return getDateDaysAfter(diff, year, month, day);
    return getDateDaysBefore(0 - diff, year, month, day);
}

private static Integer toString(Integer year * 10000 + month * 100 + monthsLeapYear[month] - before);
}
return Integer.toString(year * 10000 + month * 100 + monthsLeapYear[month] - before);
}
while (before >= monthsLeapYear[month]) {
    before -= monthsLeapYear[month];
    month--;
}
return Integer.toString(year * 10000 + month * 100 + monthsLeapYear[month] - before);
}
if (isLeapYear(year)) {
    while (before >= months[month]) {
        before -= months[month];
        month--;
    }
    year--;
    while (before >= 365 + getLeapYearCount(year)) {
        before -= 365 + getLeapYearCount(year);
        year--;
    }
    before -= monthsStacksLeapYear[month + 1];
    year--;
    month = 12;
}
return Integer.toString(year * 10000 + month * 100 + monthsLeapYear[month] - before);
}
if (before < monthsStacksLeapYear[month + 1]) {
    while (before >= monthsLeapYear[month]) {
        before -= monthsLeapYear[month];
        month--;
    }
    return Integer.toString(year * 10000 + month * 100 + monthsLeapYear[month] - before);
}
else {
    month = 12;
    year--;
    before -= monthsStacks[month + 1];
}
return Integer.toString(year * 10000 + month * 100 + months[month] - before);
}
if (isLeapYear(year)) {
    if (before < monthsStacks[month + 1]) {
        while (before >= months[month]) {
            before -= months[month];
            month--;
        }
    }
    else {
        month = 12;
        year--;
    }
}
if (month == 0) {
    year--;
    month = 12;
}
}

```

```

    private static final String threadContextDefaultValue = "-";
    private static final List<String> threadContextKeys = new ArrayList<String>(
        Arrays.asList("btokind", "mciserviceid", "std_gbl_id", "orgtr_gbl_id",
            "trz_gbl_id", "guid", "userid", "ip", "ptserviceid", "btserviceid", "btuid"));
    /**
     * Stacktrace에서 String를 추출 하는 Method
     * @param Throwable throwable exception클래스에서 catch한 Throwable e 또는 Exception e
     * @return String StackTrace String
     */
}

public class CommonUtil {

    /**
     * <ui>
     * <li> 주민 번호, 사업자 번호 format
     * <li> 금액의 천단위 구분자 format, 금액에서 소숫점 구분자 format
     * <li> 날짜 format, 시간 format
     * </ui>
     */

    protected static final Logger LOGGER =
        LoggerFactory.getLogger(BigDecimalUtil.class);

    private static final String threadContextDefaultValue = "-";
    private static final List<String> threadContextKeys = new ArrayList<String>(
        Arrays.asList("btokind", "mciserviceid", "std_gbl_id", "orgtr_gbl_id",
            "trz_gbl_id", "guid", "userid", "ip", "ptserviceid", "btserviceid", "btuid"));
    /**
     * Stacktrace에서 String를 추출 하는 Method
     * @param Throwable throwable exception클래스에서 catch한 Throwable e 또는 Exception e
     * @return String StackTrace String
     */
}

```

```

    public static String makeStackTraceString(Throwable throwable) {
        StringBuilder sb = new StringBuilder();
        for (StackTraceElement element : throwable.getStackTrace()) {
            sb.append(element.toString());
            sb.append("\n");
        }
        return sb != null ? sb.toString() : "";
    }

    /**
     * 원하는 형식에 맞춰 문자열 포매팅 변환.
     * <p>
     * <pre>
     * "123-45-67890" = formatStringDelimiter("1234567890", "-", new int[] {3,2,5})
     * "123-45-6-7890102" = formatStringDelimiter("1234567890102", "-", new int[] {3,2,1,1})
     * // 나머지 문자열은 마지막 구간에 자동 포함
     * </pre>
     *
     * @param str 문자열
     * @param delimiter 사용될 문자열
     * @param num 사용될 문자열 간 길이 (마지막 요소는 나머지 전체를 포함)
     */
    public static String formatStringDelimiter(String str, String delimiter, int[] num) {
        if (StringUtil.isBlank(str)) {
            return "";
        }
        if (num != null || num.length == 1) {
            return str;
        }
        int size = num.length;
        StringBuilder stringBuilder = new StringBuilder(str.length() + size);
        int index = 0;
        int strLength = str.length();
        for (int i = 0; i < size; i++) {
            int chunkSize = num[i];
            // 마지막 요소인 경우, 남은 문자열 전체를 포함
            if (i == size - 1) {
                chunkSize = strLength - index;
            }
            // 범위를 초과하지 않도록 제한
            if (index + chunkSize > strLength) {
                chunkSize = strLength - index;
            }
            if (chunkSize > 0) {
                stringBuilder.append(str.subSequence(index, index + chunkSize));
            }
            if (i != size - 1) {
                stringBuilder.append(delimiter);
            }
            index += chunkSize;
        }
        return stringBuilder.toString();
    }
}

```

```

/**
 * 사업자번호, 주민번호 포매팅.
 * <p>
 * <pre>
 * "123456-1234567" = formatNO("1234561234567", false);
 * "123456-*****" = formatNO("1234561234567", true);
 * "123-45-6789" = formatNO("1234567890", false);
 * "123-45-*****" = formatNO("1234567890", true);
 * </pre>
 * @param no 주민번호 또는 사업자 번호
 * @return java.lang.String
 */
public static String formatNO(String no) {
    return formatNO(no, true);
}

/**
 * 사업자번호, 주민번호 포매팅.
 * <p>
 * <pre>
 * "123456-1234567" = formatNO("1234561234567", false);
 * "123456-*****" = formatNO("1234561234567", true);
 * "123-45-6789" = formatNO("1234567890", false);
 * "123-45-*****" = formatNO("1234567890", true);
 * </pre>
 * @param no 주민번호 또는 사업자 번호
 * @param mask 주민번호 뒷자리 마스크 여부
 * @return String
 */
public static String formatNO(String no, boolean mask) {
    if (StringUtil.isBlank(no)) {
        return "";
    }
    // 입력값에 포함된 하이픈 제거
    String noClean = no.trim().replace("-", "");
    StringBuffer sb = new StringBuffer(noClean);
    if (sb.length() == 10) { // 10 자리 포맷 (사업자번호)
        sb.insert(3, "-").insert(6, "-");
    }
    if (mask) {
        sb.replace(7, 12, "*****"); // 뒷 5 자리 마스크 (67890 → ***** )
    }
    } else if (sb.length() == 13) {
        if (sb.charAt(0) == '0' && sb.charAt(6) == '0' && sb.charAt(7) == '0') { // 13자리 포맷
            sb.deleteCharAt(0);
            sb.deleteCharAt(5);
            sb.deleteCharAt(5);
            sb.insert(3, "-").insert(6, "-");
        } else { // 7개인사업자 주민번호
            sb.insert(6, "-");
        }
        sb.replace(7, 14, "*****");
    }
    }
    return sb.toString();
}

```

```

/**
 * 금액 문자열을 comma 표시 여부와 소수점 표시 여부를 확인하여 format된 문자열로 반환한다.
 * <u>
 * <li>금액 문자열이 음수(-)이면 반화된 값에도 음수(-)가 붙는다.</li>
 * <li>금액 문자열에 소수점이 있고 point 값이 있다면 톨 중 큰 숫자만을 소수점 자리가
 * 나온다.</li>
 * <li>금액 문자열에 소수점이 없고 point 값이 0이라면 금액 문자열의 소수점이하가 0이면
 * 반사처리한다. 하지만 값이 있다면 원사하지 않는다.</li>
 * </u>
 * <pre>
 * "121,564,899.231" = CommonUtil.formatMoney("0121564899.231", true, 0)
 * "121,564,899.231" = CommonUtil.formatMoney("0121564899.231", true, 2)
 * "121564899.231" = CommonUtil.formatMoney("0121564899.231", false, 2)
 * "121,564,899.2310" = CommonUtil.formatMoney("0121564899.231", true, 4)
 * "121564899.2310" = CommonUtil.formatMoney("0121564899.231", false, 4)
 * "121,564,899.2310" = CommonUtil.formatMoney("-0121564899.231", true, 4)
 * "-121,564,899.2310" = CommonUtil.formatMoney("-0121564899.231", true, 4)
 * "-121564899.2310" = CommonUtil.formatMoney("-0121564899.231", false, 4)
 * </pre>
 * @param money 금액
 * @param comma 금액 표시 여부
 * @param point 소수점의 자리수
 * @return String
 */
public static String formatMoney(String money, boolean comma, int pointVal) {
    int point = pointVal;
    StringBuilder pattern = new StringBuilder();
    if (comma) {
        pattern.append("#,##0");
    } else {
        pattern.append("###0");
    }
    int index = money.lastIndexOf('.');
    if (point < 1) {
        pattern.append("#");
    } else {
        index = money.length() - (index + 1);
        pattern.append(StringUtil.repeat('#', Math.max(index, 1)));
    }
    if (index > 0) {
        pattern.append("#");
    }
    index = money.length() - (index + 1);
    point = NumberUtils.max(index, point, 1);
    pattern.append(StringUtil.repeat('0', point));
    }
    DecimalFormat decimalFormat = new DecimalFormat(pattern.toString());
    double lamt = Double.parseDouble(money.trim());
    return decimalFormat.format(lamt);
}

public static final String DEFAULT_DELIMITER_DATE = ".";

/**
 * 날짜 8자리를 구분자(.)를 붙여서 반환.
 * <p>
 * <pre>
 * "2013.01.31" = formatDate("20130131", DEFAULT_DELIMITER_DATE);
 * "2013/01/31" = formatDate("20130131", "/");
 * </pre>
 */

```

```

    /**
     * 특수 문자 포함여부 체크
     * <p>
     * <pre>
    */
    private static final String DEFAULT_SPECIAL_KEY = "!;*&^%$#@~;|>+";

    /**
     * 특수 문자열
     */
    public static boolean isSpecialKey(String str) {
        return isSpecialKey(str, null);
    }

    /**
     * @param str 문자 조합
     * @return boolean
     */
    /**
     * <pre>
     * 지정된 key가 없으면 Character제출 특수문자 체크하고 key가 있으면 해당 키를 체크
     * <p>
     * 특수 문자 포함여부 체크.
     */
    }

    str.substring(4, 6);
    return str.substring(0, 2) + delimiter + str.substring(2, 4) + delimiter +
    }
    return str;
    if (str.length() != 6) {
    }
    return "";
    if (StringUtil.isBlank(str)) {
    public static String formatTime(String str, String delimiter) {
    }
    /**
     * @param str (HH:MM:SS)
     * @return String (HH:MM:SS)
     */
    /**
     * <pre>
     * "16:45:55" = formatTime("164555", DEFAULT_DELIMITER_TIME);
     * "16-45-55" = formatTime("164555", "-");
     * <p>
     * 시분초 6자리를 구분자(:)를 넣어서 반환.
     */
    public static final String DEFAULT_DELIMITER_TIME = ":";

    }
    str.substring(6, 8);
    return str.substring(0, 4) + delimiter + str.substring(4, 6) + delimiter +
    }
    return "";
    if (("00000000".equals(str)) || ("".equals(str))) {
    }
    return str;
    if (str.length() != 8) {
    }
    return "";
    if (StringUtil.isBlank(str)) {
    public static String formatDate(String str, String delimiter) {
    }
    /**
     * @param str (yyyymmdd)
     * @return String
     */

```